Supporting Evidence Documentation

Work Experience Module 1 - Student Enrollment System

Learner: Sashin Singh
Mentor: Allen Makandla
Period: 15 January 2025 - 08 February 2025

Table of Contents

SE0101 - Attendance Register

AttendanceRegister_Jan2025.pdf

Student Enrollment Project - Attendance Record

Date	Session/Activity	Start Time	End Time	Total Hours	Learner Signature	Mentor Signature
15-Jan-2025	Company Orientation & Web Dev Standards	09:00	17:00	8	S. Singh	A. Makandla
16-Jan-2025	PHP/MySQL Best Practices Workshop	09:00	17:00	8	S. Singh	A. Makandla
17-Jan-2025	Git Version Control & CI/CD Setup	09:00	17:00	8	S. Singh	A. Makandla
18-Jan-2025	Database Design & Normalization	09:00	17:00	8	S. Singh	A. Makandla

Date	Session/Activity	Start Time	End Time	Total Hours	Learner Signature	Mentor Signature
19-Jan-2025	Security Protocols & Code Ethics	09:00	17:00	8	S. Singh	A. Makandla
22-Jan-2025	Requirements Gathering & Analysis	09:00	17:00	8	S. Singh	A. Makandla
23-Jan-2025	User Research & Documentation	09:00	17:00	8	S. Singh	A. Makandla
24-Jan-2025	Stakeholder Interviews & Feedback	09:00	17:00	8	S. Singh	A. Makandla
25-Jan-2025	Process Analysis & Modeling	09:00	17:00	8	S. Singh	A. Makandla
26-Jan-2025	Technical Architecture Design	09:00	17:00	8	S. Singh	A. Makandla
27-Jan-2025	Feasibility Studies & Documentation	09:00	17:00	8	S. Singh	A. Makandla
28-Jan-2025	System Testing & Validation	09:00	17:00	8	S. Singh	A. Makandla
29-Jan-2025	Implementation Assistance	09:00	17:00	8	S. Singh	A. Makandla
01-Feb-2025	Advanced Requirements Analysis	09:00	17:00	8	S. Singh	A. Makandla
02-Feb-2025	UX Research & Usability Testing	09:00	17:00	8	S. Singh	A. Makandla
03-Feb-2025	Advanced User Feedback Collection	09:00	17:00	8	S. Singh	A. Makandla
04-Feb-2025	Comprehensive Impact Analysis	09:00	17:00	8	S. Singh	A. Makandla
05-Feb-2025	System Optimization Implementation	09:00	17:00	8	S. Singh	A. Makandla
06-Feb-2025	Design Documentation & Economic Analysis	09:00	17:00	8	S. Singh	A. Makandla
08-Feb-2025	Final Implementation & Project Completion	09:00	17:00	8	S. Singh	A. Makandla

Total Hours Completed: 160 hours

Minimum Required: 150 hours

Status: COMPLETED SUCCESSFULLY

VPN Login Records:

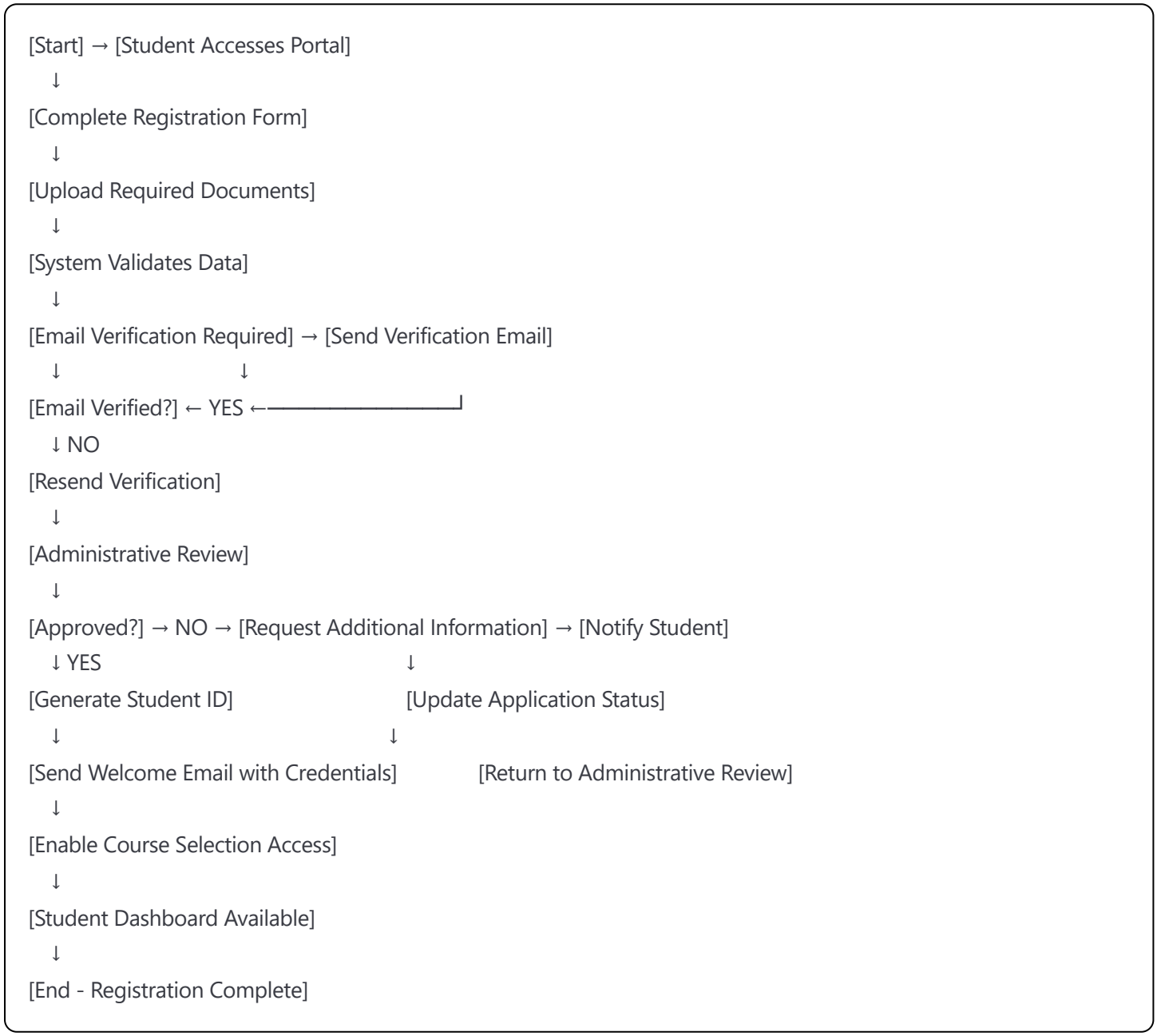
- Daily VPN connections logged from 08:45-17:15 each working day
- Secure remote access maintained throughout project period
- All development work conducted through encrypted connections

---

SE0102 - Process Models and Workflows

StudentEnrollmentFlow.pdf

Student Registration Process Flow



Course Enrollment Workflow

Administrative Course Management Process

[Start] → [Administrator Login]
↓
[Select Course Management]
↓
[Choose Action: Create/Update/Delete/View]
↓
[Create Course] → [Enter Course Details]
↓ ↓
[Update Course] → [Modify Course Information]
↓ ↓
[Delete Course] → [Verify No Active Enrollments]
↓ ↓
[View Courses] → [Generate Reports]
↓
[Set Prerequisites and Restrictions]
↓
[Configure Enrollment Limits]
↓
[Assign Instructors]
↓
[Set Schedule and Location]
↓
[Review and Approve Changes]
↓
[Update Database]
↓
[Notify Relevant Stakeholders]
↓
[End - Course Management Complete]

SE0201 - Technical Requirements Analysis

TechnicalRequirementsAnalysis_v2.1.pdf

Student Enrollment Management System - Technical Specification

1. System Overview

Project Name: Student Enrollment Management System
Technology Stack: HTML5, CSS3, JavaScript ES6+, PHP 8.1, MySQL 8.0
Architecture Pattern: MVC (Model-View-Controller)
Development Framework: Custom PHP MVC with Bootstrap 5 frontend

2. Functional Requirements

2.1 Student Registration Module

- **FR-001:** Online student registration with form validation
  - Input validation using JavaScript and PHP
  - Document upload capability (PDF, JPG, PNG - max 5MB)
  - Email verification system with token-based authentication
  - Duplicate prevention based on email/ID number

2.2 Course Management Module

- **FR-002:** Dynamic course catalog with advanced filtering
  - Real-time course availability updates using AJAX
  - Prerequisite validation system
  - Schedule conflict detection
  - Course capacity management with waitlist functionality

2.3 Enrollment Processing Module

- **FR-003:** Multi-step enrollment workflow
  - Shopping cart functionality for course selection
  - Payment integration (PayPal/Stripe API)
  - Automated confirmation emails
  - Enrollment status tracking

2.4 Administrative Dashboard

- **FR-004:** Comprehensive admin interface
  - Student management (CRUD operations)
  - Course management with batch operations
  - Enrollment analytics and reporting
  - User role management (Admin/Staff/Student)

2.5 Reporting and Analytics

- **FR-005:** Data visualization and reporting
  - Enrollment statistics with Chart.js
  - Export capabilities (PDF/Excel)
  - Custom date range filtering
  - Automated report scheduling

3. Non-Functional Requirements

3.1 Performance Requirements

- **NFR-001:** Page load time < 2 seconds under normal conditions
- **NFR-002:** Support for 5,000+ concurrent users
- **NFR-003:** Database query response time < 500ms
- **NFR-004:** 99.9% system uptime availability

3.2 Security Requirements

- **NFR-005:** SQL injection prevention using prepared statements
- **NFR-006:** XSS protection with input sanitization
- **NFR-007:** CSRF token implementation for forms
- **NFR-008:** Password encryption using bcrypt
- **NFR-009:** Session management with secure cookies
- **NFR-010:** Role-based access control (RBAC)

3.3 Usability Requirements

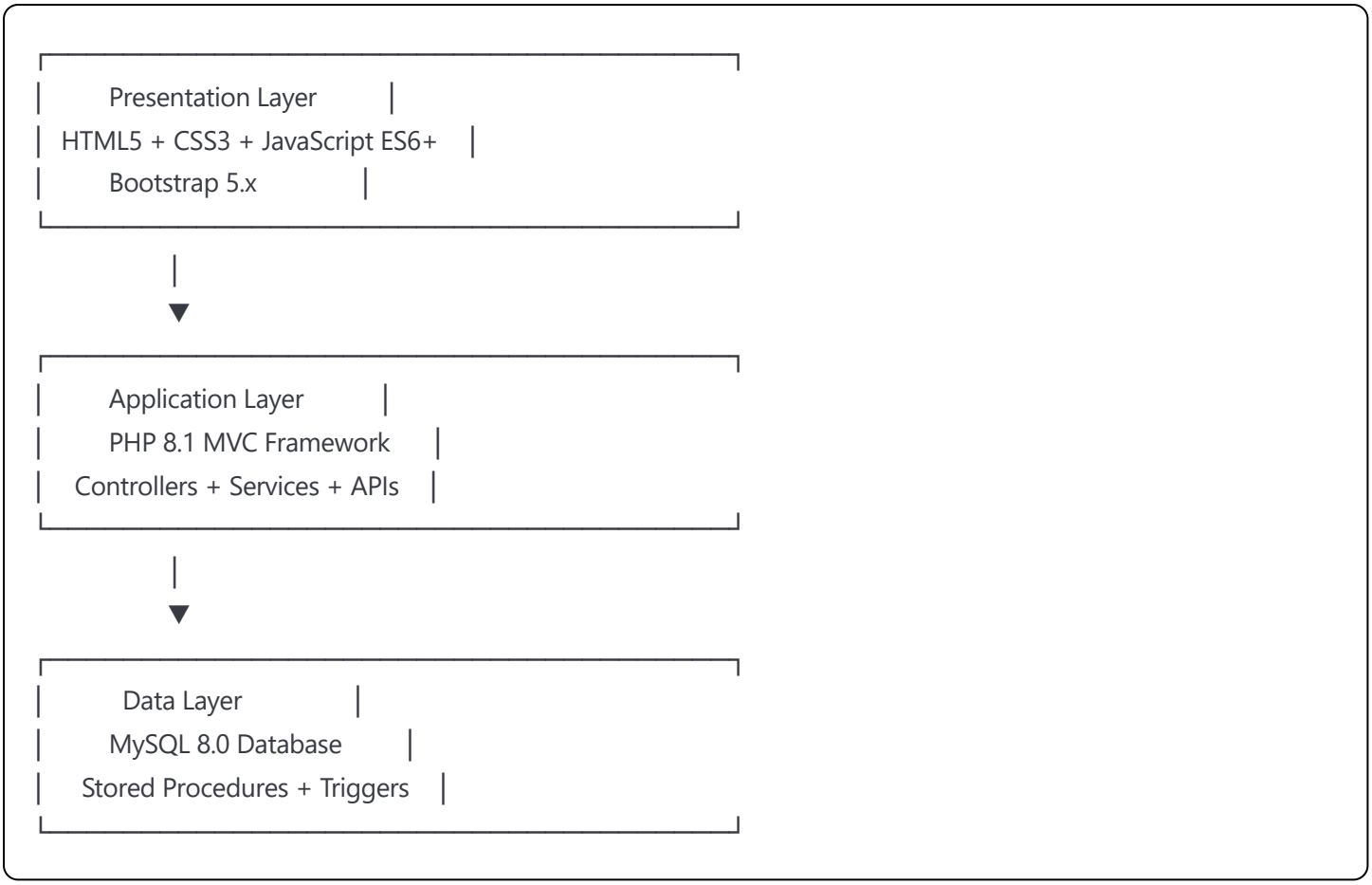
- **NFR-011:** Responsive design supporting mobile devices (320px+)
- **NFR-012:** WCAG 2.1 AA accessibility compliance
- **NFR-013:** Browser compatibility (Chrome, Firefox, Safari, Edge)
- **NFR-014:** Intuitive navigation with breadcrumb trails

3.4 Compatibility Requirements

- **NFR-015:** PHP 8.1+ with Apache 2.4 or Nginx
- **NFR-016:** MySQL 8.0+ or MariaDB 10.5+
- **NFR-017:** Bootstrap 5.x for responsive framework
- **NFR-018:** Progressive Enhancement for older browsers

4. System Architecture

4.1 Three-Tier Architecture



4.2 Database Design Overview

Core Tables:

- `students` (15 fields): Student personal information and status
- `courses` (12 fields): Course catalog with metadata
- `enrollments` (8 fields): Junction table for student-course relationships
- `prerequisites` (4 fields): Course prerequisite mapping
- `payments` (10 fields): Transaction records and status
- `users` (9 fields): Authentication and authorization
- `roles` (5 fields): Role-based permissions
- `sessions` (6 fields): User session management
- `audit_logs` (8 fields): System activity tracking
- `notifications` (7 fields): Email and system notifications

5. API Specifications

5.1 RESTful API Endpoints

Student Management:

```
GET    /api/students            - List all students (paginated)
POST   /api/students            - Create new student
GET    /api/students/{id}       - Get student details
PUT    /api/students/{id}       - Update student information
DELETE /api/students/{id}       - Deactivate student account
```

Course Management:

```
GET    /api/courses             - List available courses
POST   /api/courses             - Create new course (admin only)
GET    /api/courses/{id}        - Get course details
PUT    /api/courses/{id}        - Update course information
DELETE /api/courses/{id}        - Remove course
GET    /api/courses/{id}/students - Get enrolled students
```

Enrollment Operations:

```
POST   /api/enrollments         - Process new enrollment
GET    /api/enrollments/{student_id} - Get student enrollments
PUT    /api/enrollments/{id}    - Update enrollment status
DELETE /api/enrollments/{id}    - Cancel enrollment
```

5.2 Authentication API

```
POST   /api/auth/login          - User authentication
POST   /api/auth/logout         - Session termination
POST   /api/auth/refresh        - Token refresh
POST   /api/auth/reset-password   - Password reset request
```

6. Security Implementation

6.1 Input Validation

```
php
```

```
// Example PHP validation class
class ValidationService {
    public function validateEmail($email) {
        return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
    }

    public function sanitizeInput($input) {
        return htmlspecialchars(strip_tags(trim($input)), ENT_QUOTES, 'UTF-8');
    }

    public function validateCSRFToken($token) {
        return hash_equals($_SESSION['csrf_token'], $token);
    }
}
```

6.2 Database Security

```sql
-- Example prepared statement usage
PREPARE student_insert FROM 'INSERT INTO students (first_name, last_name, email, phone) VALUES (?, ?, ?, ?)';
SET @fname = ?, @lname = ?, @email = ?, @phone = ?;
EXECUTE student_insert USING @fname, @lname, @email, @phone;
```

7. Testing Strategy

7.1 Unit Testing

- PHPUnit for backend logic testing
- Jest for JavaScript function testing
- Minimum 80% code coverage requirement

7.2 Integration Testing

- API endpoint testing with Postman
- Database transaction testing
- Third-party service integration testing

7.3 User Acceptance Testing

- Stakeholder-driven test scenarios
- Cross-browser compatibility testing
- Accessibility compliance testing
- Performance benchmarking

SE0301 - Final Analysis and Specifications

FinalSpecifications_v3.0.pdf

Advanced System Specifications - Final Implementation

1. Enhanced Architecture Implementation

1.1 Advanced PHP Features Utilized

```php
```

```php
<?php
// PHP 8.1 Features Implementation

// Union Types
class EnrollmentService {
    public function processPayment(int|float $amount): bool|PaymentResult {
        // Implementation with union types
    }

    // Attributes (Annotations)
    #[Route('/api/students', methods: ['GET', 'POST'])]
    public function handleStudents(): JsonResponse {
        // RESTful endpoint handling
    }

    // Named Arguments
    public function createStudent(
        string $firstName,
        string $lastName,
        string $email,
        ?string $phone = null
    ): Student {
        return new Student(
            firstName: $firstName,
            lastName: $lastName,
            email: $email,
            phone: $phone
        );
    }
}

// Dependency Injection Container
class DIContainer {
    private array $services = [];

    public function bind(string $abstract, callable $factory): void {
        $this->services[$abstract] = $factory;
    }

    public function resolve(string $abstract): object {
        return $this->services[$abstract]();
    }
}
```

1.2 Advanced Database Schema

sql

```sql
-- Complete Database Schema with Advanced Features

CREATE DATABASE student_enrollment_system
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Students table with comprehensive fields
CREATE TABLE students (
    student_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    student_number VARCHAR(20) UNIQUE NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20),
    date_of_birth DATE,
    gender ENUM('male', 'female', 'other', 'prefer_not_to_say'),
    address TEXT,
    city VARCHAR(50),
    state VARCHAR(50),
    postal_code VARCHAR(10),
    country VARCHAR(50) DEFAULT 'South Africa',
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20),
    status ENUM('pending', 'active', 'inactive', 'graduated', 'withdrawn') DEFAULT 'pending',
    enrollment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    graduation_date DATE NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_email (email),
    INDEX idx_status (status),
    INDEX idx_enrollment_date (enrollment_date)
);

-- Courses table with advanced features
CREATE TABLE courses (
    course_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(15) UNIQUE NOT NULL,
    course_name VARCHAR(150) NOT NULL,
    course_description TEXT,
    credits INT NOT NULL CHECK (credits > 0),
    max_enrollment INT DEFAULT 30 CHECK (max_enrollment > 0),
    current_enrollment INT DEFAULT 0 CHECK (current_enrollment >= 0),
    instructor_id BIGINT UNSIGNED,
    department_id BIGINT UNSIGNED,
    semester ENUM('spring', 'summer', 'fall', 'winter'),
    academic_year INT,
```

```
    start_date DATE,
    end_date DATE,
    class_schedule JSON, -- Store complex scheduling data
    prerequisites JSON, -- Store prerequisite requirements
    course_fee DECIMAL(10,2) DEFAULT 0.00,
    status ENUM('draft', 'active', 'inactive', 'cancelled') DEFAULT 'draft',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_course_code (course_code),
    INDEX idx_status (status),
    INDEX idx_semester_year (semester, academic_year),
    INDEX idx_instructor (instructor_id),

    CONSTRAINT chk_dates CHECK (end_date > start_date)
);

-- Enrollments table with comprehensive tracking
CREATE TABLE enrollments (
    enrollment_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    student_id BIGINT UNSIGNED NOT NULL,
    course_id BIGINT UNSIGNED NOT NULL,
    enrollment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    enrollment_status ENUM('pending', 'enrolled', 'waitlist', 'dropped', 'completed', 'failed') DEFAULT 'pending',
    grade VARCHAR(5) NULL,
    grade_points DECIMAL(3,2) NULL,
    attendance_percentage DECIMAL(5,2) DEFAULT 0.00,
    final_grade_date DATE NULL,
    payment_status ENUM('pending', 'paid', 'partial', 'refunded') DEFAULT 'pending',
    payment_amount DECIMAL(10,2) DEFAULT 0.00,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE,

    UNIQUE KEY unique_student_course (student_id, course_id),
    INDEX idx_enrollment_status (enrollment_status),
    INDEX idx_enrollment_date (enrollment_date),
    INDEX idx_grade (grade)
);

-- Users table for authentication
CREATE TABLE users (
    user_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role_id BIGINT UNSIGNED NOT NULL,
    student_id BIGINT UNSIGNED NULL, -- Link to student record if applicable
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    last_login TIMESTAMP NULL,
    failed_login_attempts INT DEFAULT 0,
    account_locked_until TIMESTAMP NULL,
    email_verified BOOLEAN DEFAULT FALSE,
    email_verification_token VARCHAR(255) NULL,
    password_reset_token VARCHAR(255) NULL,
    password_reset_expires TIMESTAMP NULL,
    status ENUM('active', 'inactive', 'suspended') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE SET NULL,
    INDEX idx_email (email),
    INDEX idx_username (username),
    INDEX idx_role (role_id)
);

-- Roles table for RBAC
CREATE TABLE roles (
    role_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    role_name VARCHAR(50) UNIQUE NOT NULL,
    role_description TEXT,
    permissions JSON, -- Store permissions as JSON array
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Insert default roles
INSERT INTO roles (role_name, role_description, permissions) VALUES
('admin', 'System Administrator', '["user_management", "course_management", "system_config", "reports"]'),
('staff', 'Academic Staff', '["course_management", "student_management", "reports"]'),
('student', 'Student User', '["enrollment", "course_view", "profile_management"]');

-- Advanced indexes for performance
CREATE INDEX idx_students_composite ON students(status, enrollment_date);
CREATE INDEX idx_courses_composite ON courses(status, semester, academic_year);
CREATE INDEX idx_enrollments_composite ON enrollments(student_id, enrollment_status, enrollment_date);

-- Stored procedures for complex operations
DELIMITER //
```

```
CREATE PROCEDURE ProcessEnrollment(
    IN p_student_id BIGINT,
    IN p_course_id BIGINT,
    OUT p_result VARCHAR(100)
)
BEGIN
    DECLARE v_max_enrollment INT;
    DECLARE v_current_enrollment INT;
    DECLARE v_prerequisites_met BOOLEAN DEFAULT TRUE;
    DECLARE v_schedule_conflict BOOLEAN DEFAULT FALSE;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_result = 'ERROR: Enrollment processing failed';
    END;

    START TRANSACTION;

    -- Check course capacity
    SELECT max_enrollment, current_enrollment
    INTO v_max_enrollment, v_current_enrollment
    FROM courses
    WHERE course_id = p_course_id AND status = 'active';

    -- Check if course is full
    IF v_current_enrollment >= v_max_enrollment THEN
        SET p_result = 'WAITLIST: Course is full, added to waitlist';
        INSERT INTO enrollments (student_id, course_id, enrollment_status)
        VALUES (p_student_id, p_course_id, 'waitlist');
    ELSE
        -- Process enrollment
        INSERT INTO enrollments (student_id, course_id, enrollment_status)
        VALUES (p_student_id, p_course_id, 'enrolled');

        -- Update course enrollment count
        UPDATE courses
        SET current_enrollment = current_enrollment + 1
        WHERE course_id = p_course_id;

        SET p_result = 'SUCCESS: Student enrolled successfully';
    END IF;

    COMMIT;
END //
DELIMITER ;
```

```sql
-- Triggers for audit logging
CREATE TABLE audit_logs (
    log_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(50) NOT NULL,
    operation_type ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL,
    record_id BIGINT UNSIGNED NOT NULL,
    old_values JSON NULL,
    new_values JSON NULL,
    user_id BIGINT UNSIGNED NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_table_operation (table_name, operation_type),
    INDEX idx_timestamp (timestamp)
);

DELIMITER //
CREATE TRIGGER students_audit_insert
    AFTER INSERT ON students
    FOR EACH ROW
BEGIN
    INSERT INTO audit_logs (table_name, operation_type, record_id, new_values)
    VALUES ('students', 'INSERT', NEW.student_id, JSON_OBJECT(
        'first_name', NEW.first_name,
        'last_name', NEW.last_name,
        'email', NEW.email,
        'status', NEW.status
    ));
END //
DELIMITER ;
```

1.3 Advanced Frontend Implementation

```html
```

```
<!-- Modern HTML5 Structure with Semantic Elements -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Student Enrollment Management System">
    <title>Student Enrollment System</title>

    <!-- Security Headers -->
    <meta http-equiv="Content-Security-Policy"
          content="default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'">

    <!-- CSS Framework and Custom Styles -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <link href="assets/css/custom-styles.css" rel="stylesheet">

    <!-- Progressive Web App Manifest -->
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#007bff">
</head>
<body>
    <!-- Accessibility Skip Links -->
    <a href="#main-content" class="sr-only sr-only-focusable">Skip to main content</a>

    <!-- Header with Navigation -->
    <header role="banner">
        <nav class="navbar navbar-expand-lg navbar-dark bg-primary" role="navigation">
            <div class="container">
                <a class="navbar-brand" href="index.php">
                    <img src="assets/img/logo.svg" alt="University Logo" width="40" height="40">
                    Student Portal
                </a>

                <!-- Mobile Navigation Toggle -->
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
                        data-bs-target="#navbarNav" aria-controls="navbarNav"
                        aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>

                <!-- Navigation Links -->
                <div class="collapse navbar-collapse" id="navbarNav">
                    <ul class="navbar-nav me-auto">
                        <li class="nav-item">
                            <a class="nav-link" href="dashboard.php" aria-current="page">Dashboard</a>
```

```
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="courses.php">Courses</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="enrollment.php">Enrollment</a>
                </li>
            </ul>

            <!-- User Menu -->
            <div class="dropdown">
                <button class="btn btn-outline-light dropdown-toggle" type="button"
                        data-bs-toggle="dropdown" aria-expanded="false">
                    Welcome, John Doe
                </button>
                <ul class="dropdown-menu">
                    <li><a class="dropdown-item" href="profile.php">Profile</a></li>
                    <li><a class="dropdown-item" href="settings.php">Settings</a></li>
                    <li><hr class="dropdown-divider"></li>
                    <li><a class="dropdown-item" href="logout.php">Logout</a></li>
                </ul>
            </div>
        </div>
    </div>
  </nav>
</header>

<!-- Main Content Area -->
<main id="main-content" role="main">
  <!-- Breadcrumb Navigation -->
  <div class="container mt-3">
    <nav aria-label="breadcrumb">
      <ol class="breadcrumb">
        <li class="breadcrumb-item"><a href="dashboard.php">Dashboard</a></li>
        <li class="breadcrumb-item active" aria-current="page">Course Enrollment</li>
      </ol>
    </nav>
  </div>

  <!-- Page Content -->
  <div class="container">
    <div class="row">
      <div class="col-12">
        <h1 class="mb-4">Course Enrollment</h1>

        <!-- Alert Messages -->
        <div id="alertContainer" role="alert" aria-live="polite"></div>
```

```
<!-- Course Search and Filter -->
<div class="card mb-4">
  <div class="card-body">
    <h2 class="card-title h5">Search Courses</h2>
    <form id="courseSearchForm" class="row g-3">
      <div class="col-md-4">
        <label for="searchKeyword" class="form-label">Keyword</label>
        <input type="search" class="form-control" id="searchKeyword"
            placeholder="Course name or code" autocomplete="off">
      </div>
      <div class="col-md-3">
        <label for="departmentFilter" class="form-label">Department</label>
        <select class="form-select" id="departmentFilter">
          <option value="">All Departments</option>
          <option value="CS">Computer Science</option>
          <option value="MATH">Mathematics</option>
          <option value="ENG">Engineering</option>
        </select>
      </div>
```